Инженерия данных

Домашнее задание 4

Обеспечение отказоустойчивости средствами Apache ZooKeeper

Ансамбль из трёх серверов и смена лидера координатора расчёта из ДЗ 3

Вариант 8: «Программист, разработчик», Frontend

Выполнил: Московкин Александр Николаевич
ИСУ: 472264
Группа: J3312

Санкт-Петербург, 2026

Содержание

1. Установка и подготовка . . . 3
2. Ансамбль ZooKeeper . . . 4
   2.1. Конфигурация . . . 4
   2.2. Запуск и роли . . . 4
   2.3. Потеря одного сервера . . . 6
   2.4. Потеря двух серверов . . . 6
   2.5. Повторный запуск . . . 7
   2.6. Эфемерный узел . . . 7
3. Координатор расчёта с выбором лидера . . . 8
   3.1. Устройство . . . 8
4. Приостановка лидера без защиты (FENCING=0) . . . 9
5. Приостановка лидера с защитой по эпохе (FENCING=1) . . . 11
6. Время смены лидера . . . 12
7. Ответы на вопросы . . . 12
   7.1. Почему серверов три, а не два и не четыре . . . 12
   7.2. Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper . . . 12
8. Выводы . . . 13
9. Приложение. Репозиторий и воспроизведение . . . 14

Цель работы: развернуть отказоустойчивый ансамбль из трёх серверов ZooKeeper, проверить кворум и эфемерные узлы, встроить ZooKeeper в координатор расчёта пар навыков из ДЗ 3 (выбор лидера, чекпоинт, публикация результата), смоделировать приостановку лидера посреди расчёта и показать, что защита по эпохе не даёт устаревшему лидеру записать результат второй раз.

Итог в цифрах. Ансамбль ZooKeeper 3.9.6 из 3 серверов: без одного сервера работает (новый лидер выбран за 218-224 мс), без двух не отвечает (`ConnectionLoss`). Эфемерный узел исчезает сразу после `quit` и через 9,8 с после закрытия окна (тайм-аут сессии 10 с). Смена лидера координатора после `SIGSTOP`: 11,57 с в этапе 3 и 11,75 с в этапе 4, в пяти запусках от 10,47 до 11,85 с (среднее 11,37 с). Без защиты: две строки «записал результат», указатель с `dataVersion = 2`. С защитой: `BadVersionError`, указатель с `dataVersion = 1` от `coord-B`. Во всех запусках 23 204 пары, 121 358 вхождений, `javascript + typescript` = 1 654, SHA-256 `k2_pairs.csv` совпал с ДЗ 3 (`f073918f...`).

1. Установка и подготовка

Таблица 1. Стенд

Компонент	Значение
Java	OpenJDK 17.0.21, `JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64`
ZooKeeper	3.9.6, архив `apache-zookeeper-3.9.6-bin.tar.gz`, три копии в `/opt/hw4-zk/zk1..zk3`
Клиенты	`zkCli.sh` из той же сборки, Python 3.13 + kazoo 2.11 для координатора
Хранилище	RustFS 1.0.1 из ДЗ 2-3, бакет `hh-lake`, объект `raw/it_vacancies_2024.csv`
Машина	Linux x86_64, 2 vCPU, 8 ГБ RAM, все процессы на одном хосте

Архив скачан с сайта Apache и лежит в репозитории в `zk/`, его SHA-512 записан в журнал этапа 1. Каждая копия распакована целиком, в её `conf/` положены свой `zoo.cfg` и общий `logback.xml`, в `data/` свой `myid`.

```
$ java -version 2>&1 | head -1
openjdk version "17.0.21.0.3" 2026-10-24
$ echo JAVA_HOME=$JAVA_HOME
JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
$ sha512sum apache-zookeeper-3.9.6-bin.tar.gz
e999626df06de30dc8bb53bb51da9bb786b1658406adafc6b92f104d72ca25e869f70a57c83a726f8b9558cc0fb519fc20f201aa5d6da09884
apache-zookeeper-3.9.6-bin.tar.gz
$ rm -rf /opt/hw4-zk/zk1 && mkdir -p /opt/hw4-zk/zk1 && tar xzf apache-zookeeper-3.9.6-bin.tar.gz -
C /opt/hw4-zk/zk1 --strip-components=1
...
$ cp conf/zoo1.cfg /opt/hw4-zk/zk1/conf/zoo.cfg
$ cp conf/logback.xml /opt/hw4-zk/zk1/conf/logback.xml
$ mkdir -p /opt/hw4-zk/zk1/data && echo 1 > /opt/hw4-zk/zk1/data/myid
...
$ for n in 1 2 3; do echo "zk$n/data/myid = $(cat $ZK_BASE/zk$n/data/myid)"; done
zk1/data/myid = 1
zk2/data/myid = 2
zk3/data/myid = 3
$ diff <(grep ^server zk1/conf/zoo.cfg) <(grep ^server zk2/conf/zoo.cfg) && diff ... zk3 ... &&
echo "..."
строки server.N во всех трёх zoo.cfg одинаковые
$ grep -H -E "^(dataDir|clientPort|admin.serverPort)" $ZK_BASE/zk*/conf/zoo.cfg
/opt/hw4-zk/zk1/conf/zoo.cfg:dataDir=/opt/hw4-zk/zk1/data
/opt/hw4-zk/zk1/conf/zoo.cfg:clientPort=2181
/opt/hw4-zk/zk2/conf/zoo.cfg:dataDir=/opt/hw4-zk/zk2/data
/opt/hw4-zk/zk2/conf/zoo.cfg:clientPort=2182
/opt/hw4-zk/zk3/conf/zoo.cfg:dataDir=/opt/hw4-zk/zk3/data
/opt/hw4-zk/zk3/conf/zoo.cfg:clientPort=2183
```

Листинг 1. Подготовка трёх копий (`logs/stage1_setup.txt`, сокращено)

Штатные скрипты 3.9.6 считают папки `conf` и `logs` от папки своей копии, поэтому три сервера на одной машине не делят ни конфигурацию, ни журналы, ни данные. Java берётся из `JAVA_HOME`, который задаёт `zk/env.sh`. Свой `logback.xml` нужен только для журналов: в штатном включён лишь

вывод на консоль, а `zkServer.sh` при каждом старте перезаписывает файл `.out`, и история перезапусков этапа 2 терялась бы. В моём варианте сервер дописывает журнал в файл, а на консоль и в вывод `zkCli` идут только предупреждения.

2. Ансамбль ZooKeeper

2.1. Конфигурация

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/opt/hw4-zk/zk1/data
clientPort=2181
admin.serverPort=8081
4lw.commands.whitelist=srvr,stat,ruok,mntr
server.1=127.0.0.1:2888:3888
server.2=127.0.0.1:2889:3889
server.3=127.0.0.1:2890:3890
```

Листинг 2. `zk/conf/zoo1.cfg` (у zk2 и zk3 отличаются только `dataDir`, `clientPort`, `admin.serverPort`)

Таблица 2. Порты и папки трёх серверов

Сервер	clientPort	порт синхр.	порт выборов	dataDir	myid
zk1	2181	2888	3888	`/opt/hw4-zk/zk1/data`	1
zk2	2182	2889	3889	`/opt/hw4-zk/zk2/data`	2
zk3	2183	2890	3890	`/opt/hw4-zk/zk3/data`	3

Строки `server.N` одинаковые у всех трёх: каждый сервер должен знать полный состав ансамбля, чтобы считать кворум. Порты у серверов разные, потому что все три слушают один и тот же адрес 127.0.0.1. Файл `myid` говорит серверу, какая из строк `server.N` описывает его самого: по ней сервер узнаёт свои порты синхронизации и выборов. Если бы `dataDir` был общим, серверы писали бы снапшоты и журнал транзакций в одни и те же файлы и затирали `myid` друг друга.

`tickTime = 2000` задаёт и границы тайм-аута сессии: от 2 × tickTime = 4 с до 20 × tickTime = 40 с. Запрошенные координатором 10 с попадают в этот диапазон, и сервер подтвердил их без изменений (в журналах координаторов строка «согласованный тайм-аут 10 с»).

2.2. Запуск и роли

Серверы запускаются штатной командой `zkServer.sh start`, роль смотрится `zkServer.sh status` (внутри это 4-буквенная команда `srvr`). Все выводы этапа лежат в `logs/stage2_ensemble.txt`, журналы самих серверов в `logs/zk-server/zk1.log ... zk3.log`.

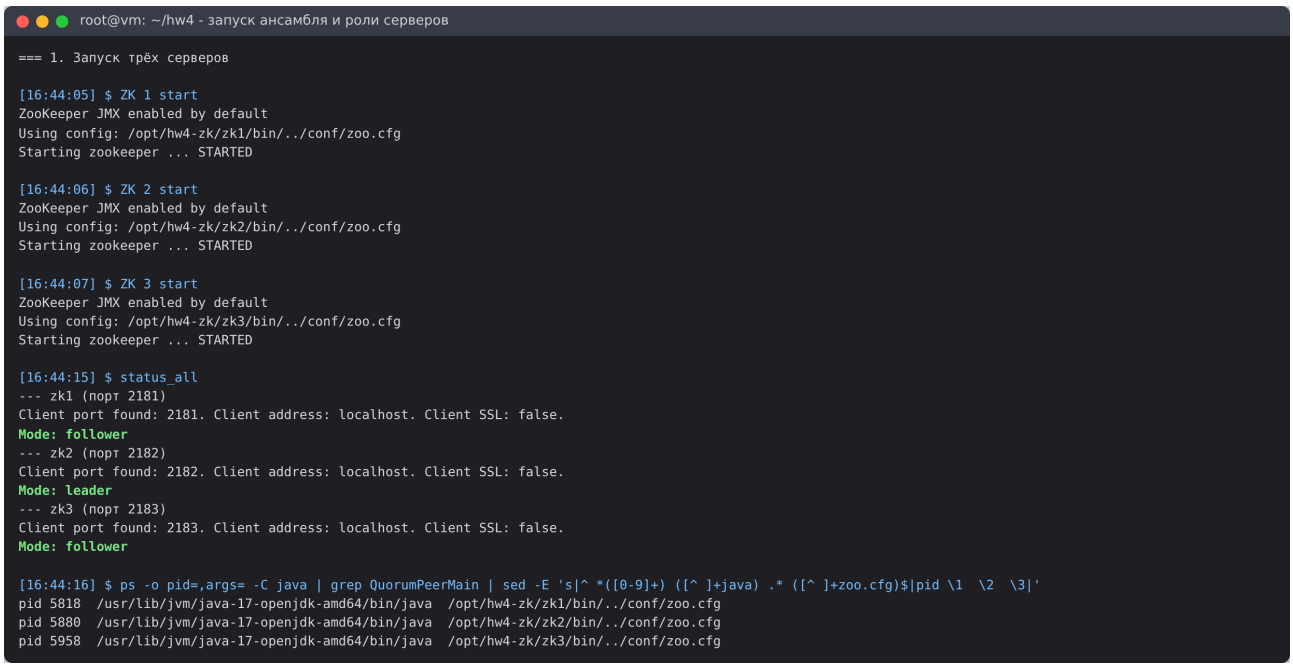


Рис. 1. Запуск трёх серверов: zk2 лидер, zk1 и zk3 последователи, все три работают на Java 17

```
[16:44:16] $ for p in 2181 2182 2183; do echo "--- $p"; echo srvr | nc -q 2 127.0.0.1 $p | grep -E
"version|Mode|Zxid|Node count"; done
--- 2181
Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UTC
Zxid: 0x0
Mode: follower
--- 2182
Zxid: 0x100000000
Mode: leader
--- 2183
Zxid: 0x100000000
Mode: follower
[16:44:22] $ CLI $ZK_ALL create /hw4-ensemble "created with 3 servers"
Created /hw4-ensemble
[16:44:23] $ CLI 127.0.0.1:2183 get /hw4-ensemble
created with 3 servers
```

Листинг 3. Роли через `srvr` (`logs/stage2_ensemble.txt`, сокращено)

Старшие 32 бита Zxid (`0x1...`) - это эпоха Zab: номер текущего «правления» лидера. Она растёт при каждой смене лидера ансамбля (ниже в журналах видны `0x2...` и `0x3...`). Это та же идея, что и эпоха координатора в этапах 3-4, только внутри самого ZooKeeper.



Рис. 2. AdminServer трёх серверов (порты 8081-8083) в браузере: `server_state` у zk1 «leader», у zk2 и zk3 «follower» (снято в конце всех запусков)

2.3. Потеря одного сервера

Для наглядности остановлен именно лидер zk2: так проверяется не только кворум, но и выбор нового лидера.

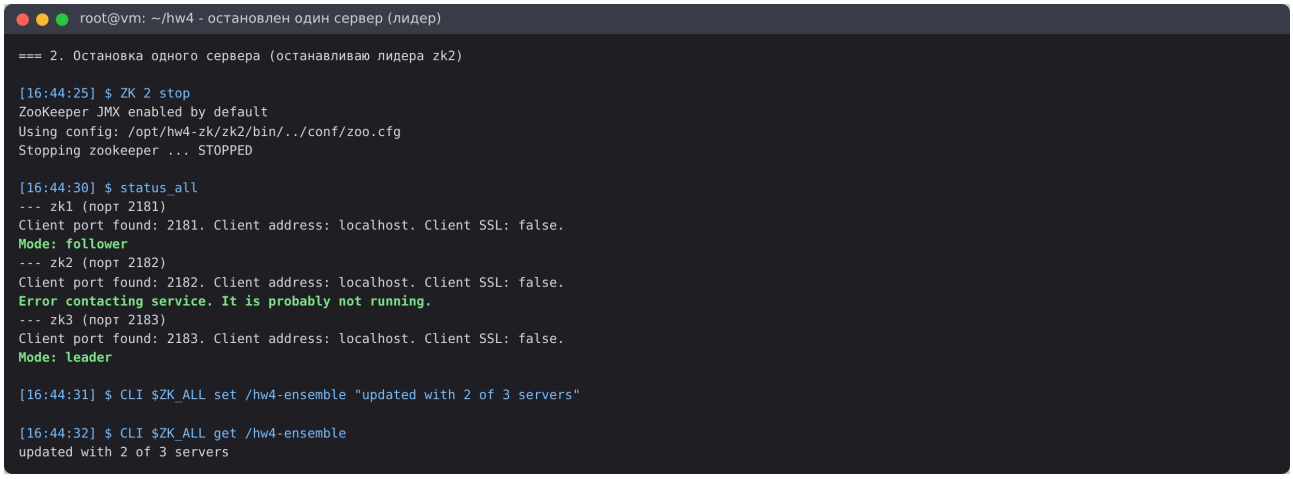


Рис. 3. Без zk2 ансамбль работает: zk3 стал лидером, запись и чтение через zkCli проходят

```
zk1.log 2026-11-10 16:44:25,514 [myid:] - WARN  o.a.z.s.q.QuorumPeer@1601] - PeerState set to
LOOKING
zk3.log 2026-11-10 16:44:25,526 [myid:] - WARN  o.a.z.s.q.QuorumPeer@1601] - PeerState set to
LOOKING
zk1.log 2026-11-10 16:44:25,738 [myid:] - INFO  o.a.z.s.q.Follower@76] - FOLLOWING - LEADER
ELECTION TOOK - 224 MS
zk3.log 2026-11-10 16:44:25,747 [myid:] - INFO  o.a.z.s.q.Leader@637] - LEADING - LEADER ELECTION
TOOK - 218 MS
```

Листинг 4. Выборы нового лидера в журналах серверов (`logs/zk-server/`)

Два сервера из трёх составляют большинство (кворум 2), поэтому они выбрали лидера за 218-224 мс, что совпадает с оценкой из лекции (порядка 200 мс), и продолжили принимать записи: `set / hw4-ensemble "updated with 2 of 3 servers"` прошёл.

2.4. Потеря двух серверов

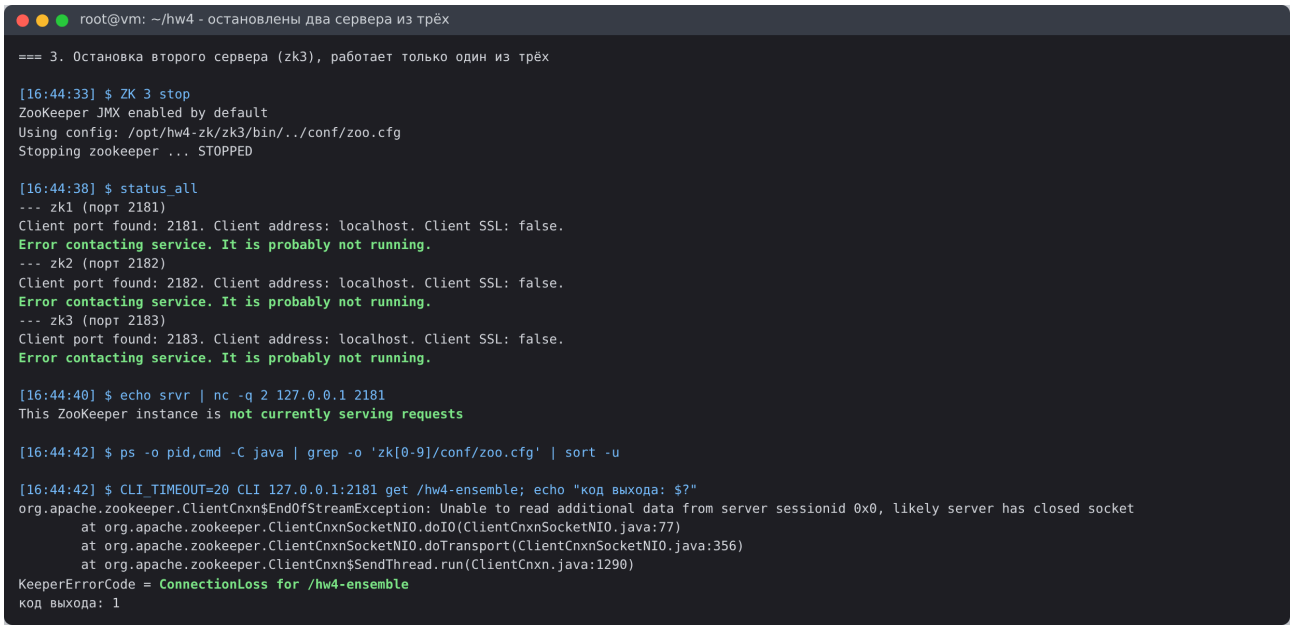


Рис. 4. Без zk2 и zk3 последний живой сервер zk1 не обслуживает запросы, клиент получает ConnectionLoss

Процесс zk1 жив и слушает порт 2181, но на `srvr` он отвечает `This ZooKeeper instance is not currently serving requests`, `zkServer.sh status` пишет `Error contacting service`, а `zkCli get` завершается с `KeeperErrorCode = ConnectionLoss`. Перед ошибкой клиент печатает предупреждение

`EndOfStreamException: ... likely server has closed socket`: сервер принимает TCP-соединение и сразу его закрывает, потому что без кворума не обслуживает клиентов. Один сервер из трёх не кворум, поэтому он не может ни стать лидером, ни следовать за кем-то. По журналу видно, как он раз за разом начинает выборы и удваивает ожидание ответа:

```
16:44:34,214 [myid:] - WARN  o.a.z.s.q.QuorumPeer@1601] - PeerState set to LOOKING
16:44:34,214 [myid:] - INFO  o.a.z.s.q.FastLeaderElection@947] - New election. My id = 1, proposed
zxid=0x200000005
16:44:34,419 [myid:] - INFO  o.a.z.s.q.FastLeaderElection@997] - Notification time out: 400 ms
16:44:34,820 [myid:] - INFO  o.a.z.s.q.FastLeaderElection@997] - Notification time out: 800 ms
16:44:35,621 [myid:] - INFO  o.a.z.s.q.FastLeaderElection@997] - Notification time out: 1600 ms
16:44:37,222 [myid:] - INFO  o.a.z.s.q.FastLeaderElection@997] - Notification time out: 3200 ms
16:44:40,425 [myid:] - INFO  o.a.z.s.q.FastLeaderElection@997] - Notification time out: 6400 ms
16:44:43,626 [myid:] - INFO  o.a.z.s.q.Leader@637] - LEADING - LEADER ELECTION TOOK - 9411 MS
```

Листинг 5. zk1 один: бесконечные выборы без кворума (`logs/zk-server/zk1.log`)

2.5. Повторный запуск

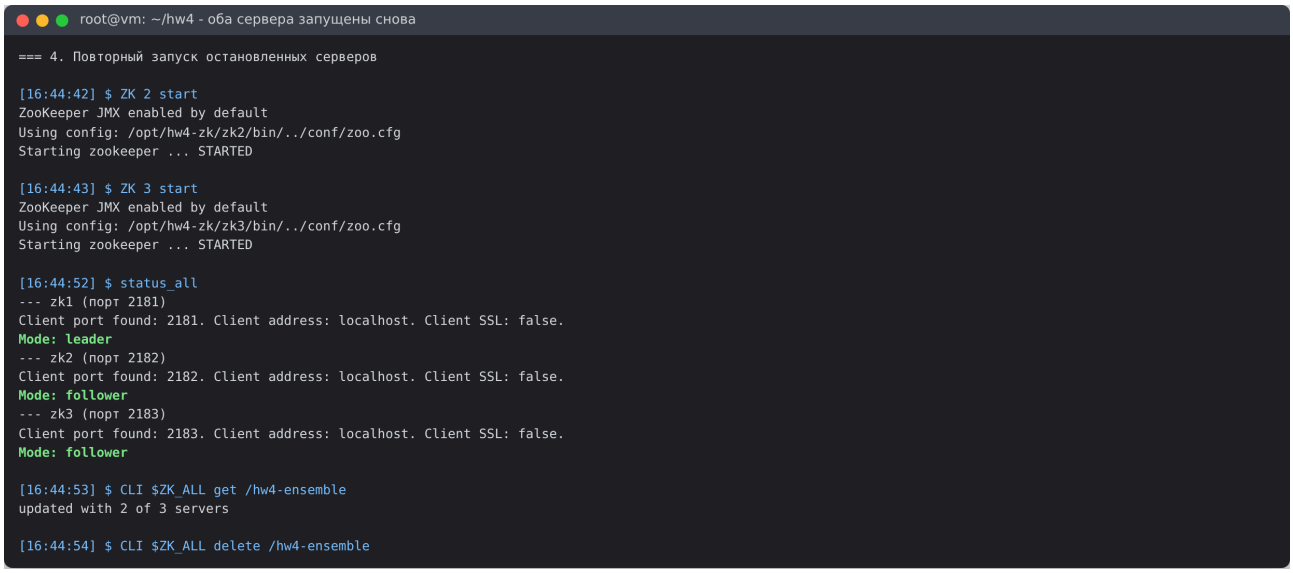


Рис. 5. После запуска zk2 и zk3 ансамбль снова из трёх: zk1 лидер, данные на месте

Как только поднялся zk2, у zk1 появился кворум, и он стал лидером (9 411 мс в журнале - это всё время, пока он был один). Узел `/hw4-ensemble` хранит значение, записанное при двух серверах: подтверждённая кворумом запись пережила остановку двух серверов из трёх.

2.6. Эфемерный узел

`logs/stage2_ephemeral.txt` получен скриптом `zk/ephemeral_demo.py`: «окно 2» - интерактивный `zkCli.sh`, команды в который подаются через stdin, «окно 1» - отдельные запуски `zkCli.sh` для проверки. Проведено два опыта.

```
[16:44:55.424] окно 2 $ zkCli.sh -timeout 10000 -server
127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183   (интерактивно)
[16:44:59.425] окно 2 > create -e /hw4-ephemeral "ephemeral from window 2"
[16:45:01.426] окно 1 $ zkCli.sh -server 127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183 stat /hw4-
ephemeral
...
ephemeralOwner = 0x200000ffd2c0001
...
[16:45:02.206] окно 1 $ zkCli.sh -server 127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183 ls /
[hw4-ephemeral, zookeeper]
[16:45:03.033] окно 2 > quit
[16:45:05.034] окно 2 закрыто командой quit, код 0
    вывод окна 2:
    | [zk: 127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183(CONNECTED) 0] Created /hw4-ephemeral
```

```
[16:45:05.034] окно 1 $ zkCli.sh -server 127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183 ls /
[zookeeper]
```

Листинг 6. Опыт А: узел создан в окне 2, окно закрыто командой `quit`

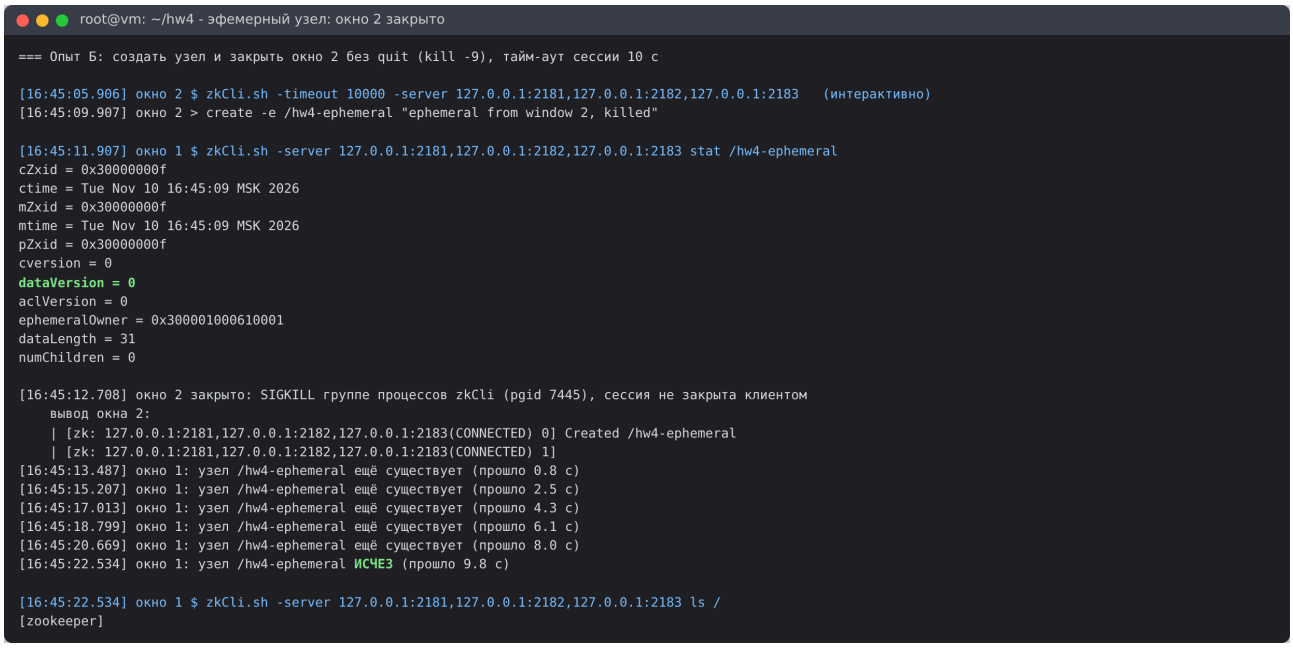


Рис. 6. Опыт Б: окно 2 закрыто без quit, узел живёт до истечения сессии и исчезает через 9,8 с

`ephemeralOwner` - номер сессии окна 2, к ней и привязан узел. При `quit` клиент сам закрывает сессию, поэтому узел пропал сразу. При закрытии окна процесс просто убит и ничего не успевает сообщить серверу, поэтому сервер ждёт тайм-аута сессии (10 с) и только потом удаляет узел. Получилось 9,8 с, чуть меньше 10: сервер отсчитывает тайм-аут не от закрытия окна, а от последнего пакета клиента, а клиент шлёт ping примерно раз в треть тайм-аута, так что часть тайм-аута истекла ещё до kill. Именно на этой задержке построена смена лидера координатора в следующих этапах: лидерство держится эфемерным узлом сессии, и запасной узнаёт о потере лидера не раньше, чем истечёт тайм-аут.

3. Координатор расчёта с выбором лидера

3.1. Устройство

`coordinator/coordinator.py` использует код ДЗ 3 без изменений (`coordinator/hw3/common.py` и `mapreduce.py`): чтение диапазона байт из `s3://hh-lake/raw/it_vacancies_2024.csv`, условие варианта 8, нормализацию навыков и формулу зарплаты. Объект делится на 51 чанк по 16 МиБ.

Таблица 3. Узлы координатора

Узел ZooKeeper	Назначение
`/hw4/<run>/election`	рецепт `Election` из kazoo: каждый координатор создаёт эфемерный последовательный узел, лидер тот, у кого номер меньше; запасной ставит watch на узел перед своим
`/hw4/<run>/epoch`	номер эпохи: новый лидер делает `set` этого узла, его `dataVersion` и есть эпоха (1 у первого лидера, 2 у второго)
`/hw4/<run>/checkpoint/chunk-NNNNN`	чекпоинт: узел на каждый посчитанный чанк, частичный результат лежит в `s3://hh-lake/hw4/<run>/partials/`
`/hw4/<run>/result`	указатель на итоговый CSV: путь, SHA-256, контрольные числа, автор и эпоха. Создаётся с версией 0

Порядок работы лидера: стать лидером, увеличить эпоху, прочитать чекпоинт, досчитать несделанные чанки (каждый: map из ДЗ 3, запись частичного результата в S3, узел в чекпоинте), свести частичные результаты в `k2_pairs.csv` того же формата, что в ДЗ 3, сравнить SHA-256 с `results/mapreduce/chunk64/k2_pairs.csv` из ДЗ 3, записать CSV в S3 и опубликовать указатель. Отметка чанка

создаётся через `create`, повторная отметка даёт `NodeExistsError` и игнорируется, а частичный результат детерминирован, поэтому повтор чанка безвреден. Единственная неидемпотентная операция - публикация указателя.

```
def lead(self):
    self.log.info("стал лидером")
    estat = self.zk_call(self.zk.set, f"{self.base}/epoch", self.name.encode())
    self.epoch = estat.version                      # номер эпохи = версия узла epoch
    ...

def publish(self, key, sha, uniq, occ):
    path = f"{self.base}/result"
    if not FENCING:                                 # без защиты: кто последний, тот и записал
        stat = self.zk_call(self.zk.set, path, payload)
        self.log.info(f"записал результат: ... версия указателя {stat.version}")
        return
    tx = self.zk.transaction()                      # FENCING=1: атомарно
    tx.check(f"{self.base}/epoch", version=self.epoch)   # 1) моя эпоха всё ещё текущая?
    tx.set_data(path, payload)                      #    2) только тогда меняем указатель
    res = self.zk_call(tx.commit)
    if isinstance(res[0], BadVersionError):
        self.log.info(f"BadVersionError: запись результата отклонена, моя эпоха {self.epoch}, ...")
        return
```

Листинг 7. Публикация результата с защитой по эпохе и без неё (`coordinator/coordinator.py`)

Сценарий `coordinator/scenario.py` запускает `coord-A`, через секунду `coord-B` (у обоих тайм-аут сессии 10 с), ждёт, пока в чекпоинте будет 24 чанка из 51, и отправляет лидеру `SIGSTOP`. Так моделируется долгая пауза процесса (сборка мусора, свопинг, зависшая виртуальная машина): процесс не падает, а просто перестаёт работать, в том числе перестаёт слать heartbeat в ZooKeeper. Через 40 с (к этому моменту запасной уже закончил) сценарий отправляет `SIGCONT`, дожидается обоих процессов и печатает итог. Время смены лидера считается от момента `SIGSTOP` в журнале сценария до строки «стал лидером» в журнале запасного; часы общие, все процессы на одной машине.

4. Приостановка лидера без защиты (FENCING=0)

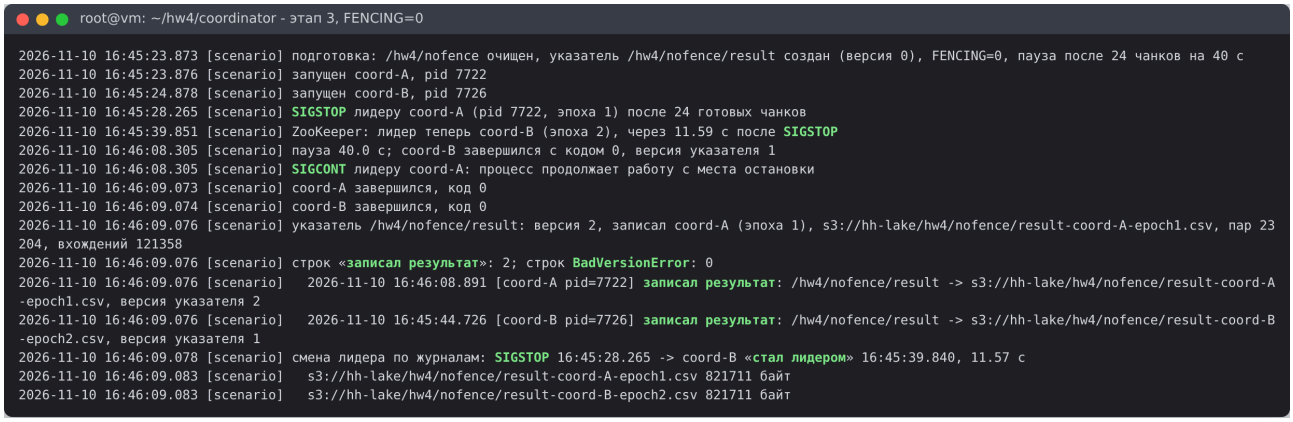


Рис. 7. Этап 3: две строки «записал результат», указатель с версией 2 от вернувшегося coord-A

```
16:45:25.291 [coord-B pid=7726] сессия 0x100000ffd340006, согласованный тайм-аут 10 с
16:45:25.302 [coord-B pid=7726] жду лидерства (рецепт Election, эфемерный последовательный узел)
16:45:39.840 [coord-B pid=7726] стал лидером
16:45:39.845 [coord-B pid=7726] эпоха 2 (версия узла /hw4/nofence/epoch)
16:45:39.858 [coord-B pid=7726] чекпоинт: готово 24 из 51 чанков, продолжаю с чанка 24
16:45:40.052 [coord-B pid=7726] чанк 24 готов (25/51): строк 4541, вакансий варианта 150, пар 1743,
0.19 с
...
16:45:44.265 [coord-B pid=7726] чанк 50 готов (51/51): строк 2304, вакансий варианта 65, пар 677,
0.10 с
16:45:44.696 [coord-B pid=7726] reduce: 0.43 с; контрольные числа: уникальных пар 23204, вхождений
121358, javascript + typescript = 1654
```

```
16:45:44.696 [coord-B pid=7726] sha256 k2_pairs.csv f073918f0709a7cb..., ДЗ 3 f073918f0709a7cb... -
> СОВПАДАЕТ с ДЗ 3
16:45:44.726 [coord-B pid=7726] записал результат: /hw4/nofence/result -> s3://hh-lake/hw4/nofence/
result-coord-B-epoch2.csv, версия указателя 1
```

Листинг 8. Журнал запасного координатора (`logs/nofence/coord-B.log`, сокращено)

```
16:45:24.210 [coord-A pid=7722] стал лидером
16:45:24.216 [coord-A pid=7722] эпоха 1 (версия узла /hw4/nofence/epoch)
16:45:24.225 [coord-A pid=7722] чекпоинт пуст: начинаю расчёт с нуля, 51 чанков по 16 МиБ
16:45:28.232 [coord-A pid=7722] чанк 23 готов (24/51): строк 4612, вакансий варианта 165, пар 1245,
0.15 с
                                    <- здесь SIGSTOP (16:45:28.265) посреди чанка 24, 40 с тишины,
SIGCONT (16:46:08.305)
16:46:08.311 [coord-A pid=7722] связь с ZooKeeper прервана (SUSPENDED)
16:46:08.329 [coord-A pid=7722] сессия ZooKeeper потеряна (LOST), эфемерные узлы удалены
16:46:08.379 [coord-A pid=7722] подключён к ZooKeeper (CONNECTED)
16:46:08.442 [coord-A pid=7722] чанк 24: отметка в чекпоинте уже есть
16:46:08.443 [coord-A pid=7722] чанк 24 готов (51/51): строк 4541, вакансий варианта 150, пар 1743,
40.21 с
16:46:08.467 [coord-A pid=7722] чанки 25-50 (26 шт.) уже в чекпоинте, пропускаю
16:46:08.865 [coord-A pid=7722] reduce: 0.40 с; контрольные числа: уникальных пар 23204, вхождений
121358, javascript + typescript = 1654
16:46:08.865 [coord-A pid=7722] sha256 k2_pairs.csv f073918f0709a7cb..., ДЗ 3 f073918f0709a7cb... -
> СОВПАДАЕТ с ДЗ 3
16:46:08.891 [coord-A pid=7722] записал результат: /hw4/nofence/result -> s3://hh-lake/hw4/nofence/
result-coord-A-epoch1.csv, версия указателя 2
```

Листинг 9. Журнал приостановленного лидера (`logs/nofence/coord-A.log`, сокращено)

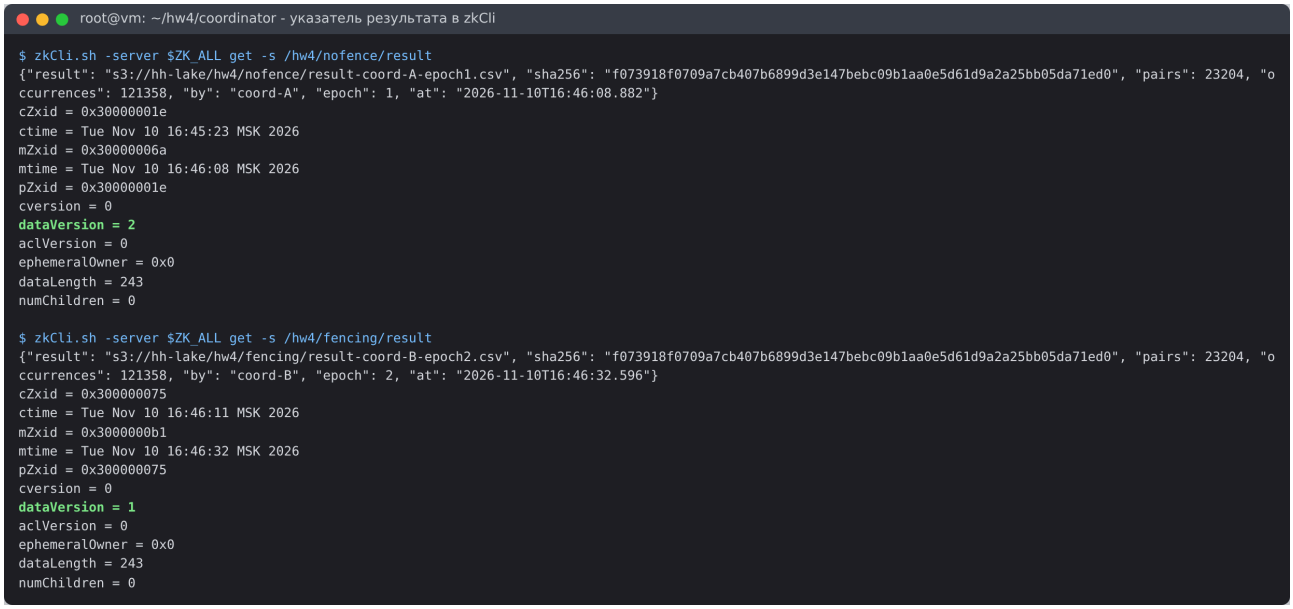


Рис. 8. Указатель глазами zkCli: в этапе 3 `dataVersion = 2` и автор coord-A эпохи 1, в этапе 4 `dataVersion = 1` и автор coord-B эпохи 2

Что произошло. Через 11,57 с после `SIGSTOP` сервер ZooKeeper закрыл сессию лидера по тайм-ауту, удалил его эфемерный узел в `election`, watch разбудил `coord-B`, тот стал лидером эпохи 2, увидел в чекпоинте 24 готовых чанка и досчитал оставшиеся 27 за 4,4 с. Результат запасного совпал с ДЗ 3 побайтово. После `SIGCONT` процесс `coord-A` продолжил работу с той же строки кода, где его остановили. kazoo сообщил ему о потере сессии (`LOST`) и тут же открыл новую, но код лидера об этом не спрашивает: он закончил свой чанк, увидел, что остальные уже отмечены, свёл результат и записал указатель второй раз. Итог: две строки «записал результат», `dataVersion = 2`, а в указателе стоит автор `coord-A` с устаревшей эпохой 1. В этом опыте содержимое CSV у обоих совпало, потому что расчёт детерминирован, но указатель уже указывает на результат процесса, который не является лидером. Если бы данные за время паузы изменились, «зомби» перезаписал бы свежий результат старым.

5. Приостановка лидера с защитой по эпохе (FENCING=1)

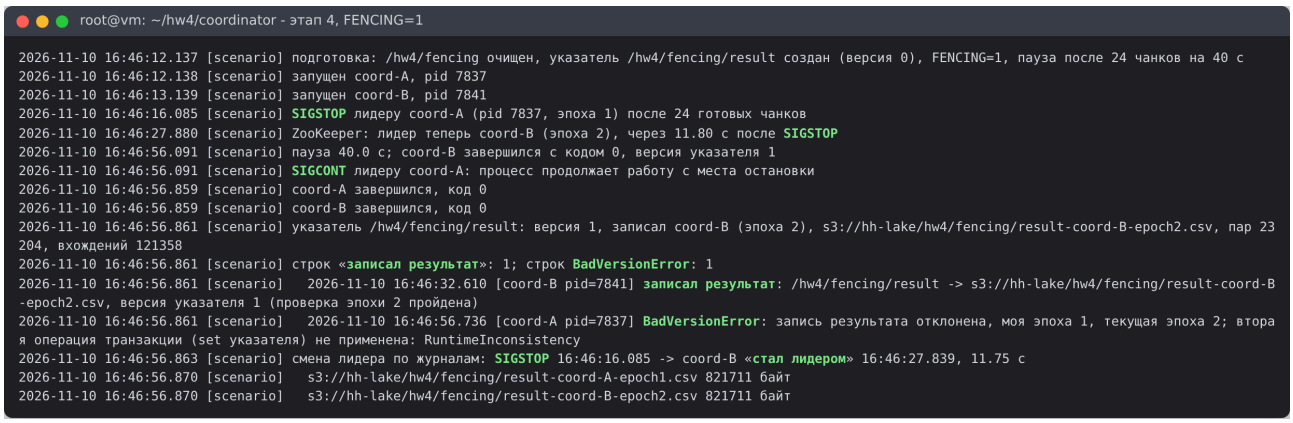


Рис. 9. Этап 4: одна строка «записал результат» от coord-B и BadVersionError у вернувшегося coord-A

```
coord-B 16:46:27.839 стал лидером
coord-B 16:46:27.843 эпоха 2 (версия узла /hw4/fencing/epoch)
coord-B 16:46:27.851 чекпоинт: готово 24 из 51 чанков, продолжаю с чанка 24
coord-B 16:46:32.570 sha256 k2_pairs.csv f073918f0709a7cb..., ДЗ 3 f073918f0709a7cb... -> СОВПАДАЕТ
с ДЗ 3
coord-B 16:46:32.610 записал результат: /hw4/fencing/result -> s3://hh-lake/hw4/fencing/result-
coord-B-epoch2.csv,
                     версия указателя 1 (проверка эпохи 2 пройдена)
coord-A 16:46:56.101 сессия ZooKeeper потеряна (LOST), эфемерные узлы удалены
coord-A 16:46:56.126 подключён к ZooKeeper (CONNECTED)
coord-A 16:46:56.700 sha256 k2_pairs.csv f073918f0709a7cb..., ДЗ 3 f073918f0709a7cb... -> СОВПАДАЕТ
с ДЗ 3
coord-A 16:46:56.736 BadVersionError: запись результата отклонена, моя эпоха 1, текущая эпоха 2;
                     вторая операция транзакции (set указателя) не применена: RuntimeInconsistency
```

Листинг 10. Ключевые строки журналов этапа 4 (`logs/fencing/`)

Транзакция `check(epoch, version=1) + set(result)` выполняется атомарно: условие проверяется в момент применения записи, и если оно ложно, не применяется ни одна операция. Вернувшийся `coord-A` думает, что он лидер эпохи 1, но версия узла `epoch` уже 2, поэтому первая операция даёт `BadVersionError`, а `set` указателя откатывается. Указатель остался в версии 1 с автором `coord-B` (Рис. 8), строка «записал результат» одна. Расчёт при этом тоже совпал с ДЗ 3.

При этом файл `result-coord-A-epoch1.csv` в S3 всё равно появился (Рис. 10), потому что RustFS эпоху не проверяет. Его никто не читает, на него не указывает указатель, но это хорошо показывает границу защиты: защищена только та запись, получатель которой проверяет эпоху.

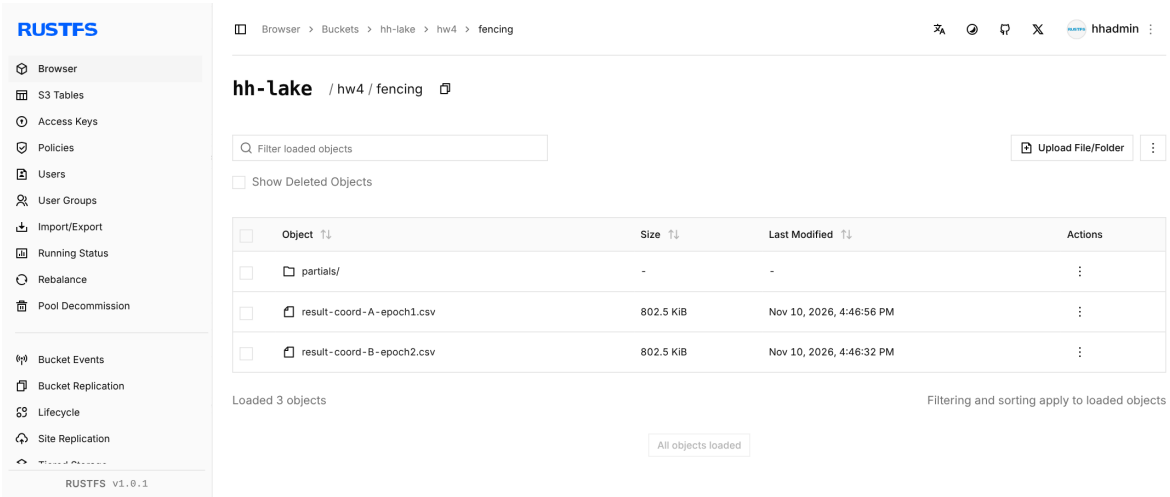


Рис. 10. `s3://hh-lake/hw4/fencing/`: частичные результаты и два CSV, но указатель ведёт только на результат coord-B

6. Время смены лидера

Таблица 4. Время от приостановки лидера до момента, когда запасной стал лидером (`logs/*/summary.json`)

Запуск	FENCING	SIGSTOP	«стал лидером»	время, с	указатель
`nofence` (этап 3)	0	16:45:28.265	16:45:39.840	11,57	v2, coord-A
`fencing` (этап 4)	1	16:46:16.085	16:46:27.839	11,75	v1, coord-B
`failover1`	1	16:47:03.983	16:47:15.838	11,85	v1, coord-B
`failover2`	1	16:47:49.365	16:47:59.836	10,47	v1, coord-B
`failover3`	1	16:48:34.656	16:48:45.838	11,18	v1, coord-B
среднее				**11,37**	

Время складывается из тайм-аута сессии 10 с и задержки, с которой лидер ансамбля ZooKeeper замечает истечение сессии. Сервер проверяет сессии пачками по tickTime (2 с), поэтому истечение фиксируется на 0-2 с позже, чем сессия формально кончилась. Это хорошо видно по столбцу «стал лидером»: во всех пяти запусках запасной становится лидером в момент с одинаковой дробной частью секунды (xx.836-xx.840) и в одной и той же фазе двухсекундного цикла (всегда нечётная секунда). Это моменты, когда лидер ансамбля обходит очередную пачку истекших сессий, а время самой паузы ни на что не влияет. Удаление эфемерного узла, уведомление watch и захват лидерства самим запасным занимают единицы миллисекунд. Отсюда разброс 10,5-11,9 с при среднем 11,4 с. Уменьшить время можно только уменьшив тайм-аут сессии, но тогда короткая пауза живого лидера (сборка мусора, нагрузка) чаще будет считаться его смертью: это компромисс между скоростью переключения и ложными переключениями.

7. Ответы на вопросы

7.1. Почему серверов три, а не два и не четыре

Ансамбль работает, пока жива строгая половина серверов плюс один: кворум из N серверов равен $\lfloor N/2 \rfloor + 1$. Это нужно, чтобы любые два кворума пересекались хотя бы в одном сервере: тогда два лидера не могут быть выбраны одновременно, а подтверждённая запись всегда есть хотя бы у одного участника следующих выборов.

- **Два сервера.** Кворум 2 из 2, значит ансамбль не переживает потерю ни одного сервера. Это хуже, чем один сервер: точек отказа стало две, а запас прочности остался нулевым. Если же разрешить работу одному из двух, при разрыве сети между ними оба решат, что второй умер, и мы получим двух лидеров.
- **Три сервера.** Кворум 2 из 3, ансамбль переживает отказ одного сервера. Ровно это показано в разделе 2: без zk2 выбран новый лидер за 218-224 мс и записи идут, без zk2 и zk3 один zk1 перестал обслуживать запросы.
- **Четыре сервера.** Кворум 3 из 4, ансамбль по-прежнему переживает только один отказ, как и три сервера. Зато каждая запись ждёт подтверждения от трёх серверов вместо двух, серверов, которые могут сломаться, стало больше, а при разрыве сети пополам (2 + 2) ни одна половина не имеет большинства, и весь ансамбль встаёт. Четвёртый сервер добавляет стоимость и не добавляет устойчивости. Следующий осмысленный размер - пять (кворум 3, переживает два отказа), поэтому число серверов почти всегда нечётное.

В моём стенде все три сервера на одной машине, поэтому от отказа самой машины ансамбль не защищает: в продакшене серверы ставят на разные машины и в разные стойки или зоны.

7.2. Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

ZooKeeper отвечает на вопрос «кто лидер сейчас», но не может помешать бывшему лидеру действовать. Между моментом, когда процесс последний раз убедился в своём лидерстве, и моментом записи может пройти сколько угодно времени: в этапах 3-4 это 40 с паузы. Всё это время процесс ничего не знает и ничего не может сделать. Когда он продолжает работу, он сразу выполняет следующую строку кода, то есть запись. Даже если бы он перед записью спросил ZooKeeper «я

ещё лидер?», между ответом и самой записью снова может случиться пауза. Проверка и действие разнесены во времени, и закрыть этот разрыв со стороны отправителя нельзя.

Кроме того, ZooKeeper вообще не видит запись: результат уходит в хранилище (S3, база данных, файл), а ZooKeeper не стоит на этом пути и не может её перехватить. Единственный участник, который видит запись в момент её применения, - получатель. Поэтому именно он сравнивает эпоху, приложенную к записи, с самой новой эпохой, которую уже видел, и отклоняет запись со старой эпохой. Так как эпоха только растёт, вернувшийся «зомби» с эпохой 1 не пройдёт проверку после того, как лидер эпохи 2 уже что-то записал или хотя бы объявил свою эпоху.

В моей реализации получатель указателя - узел ZooKeeper `/hw4/<run>/result`, поэтому проверку выполняет та же система, но в роли хранилища, а не сервиса выборов: условие `check(epoch, version=моя_эпоха)` проверяется атомарно вместе с `set` в момент применения транзакции. Опыт с файлом `result-coord-A-epoch1.csv` в RustFS показывает обратную сторону: хранилище, которое эпоху не проверяет, приняло запись зомби. Чтобы защитить и его, эпоху нужно передавать вместе с записью и проверять на стороне S3 (например, условной записью или отказом от ключей с меньшей эпохой).

8. Выводы

1. Ансамбль из трёх серверов ZooKeeper 3.9.6 собран из трёх копий с одинаковыми строками `server.N` и своими портами, `dataDir` и `myid`; запуск штатным `zkServer.sh` на Java 17, роли видны через `status` и `srvr`.
2. Без одного сервера ансамбль работает (новый лидер за 218-224 мс), без двух не отвечает: `not currently serving requests`, `ConnectionLoss`, сервер бесконечно в `LOOKING`. После запуска остановленных серверов ансамбль восстановился, данные сохранились.
3. Эфемерный узел исчезает сразу при закрытии сессии командой `quit` и через 9,8 с, если окно закрыто без `quit`: сервер ждёт тайм-аута сессии.
4. Запасной координатор становится лидером в среднем через 11,4 с после приостановки лидера (10,5-11,9 с в пяти запусках) и продолжает с чекпоинта; контрольные числа и SHA-256 результата совпали с ДЗ 3 во всех запусках.
5. Без защиты вернувшийся лидер записал результат второй раз (две строки «записал результат», указатель версии 2 от лидера устаревшей эпохи). С защитой по эпохе его запись отклонена с `BadVersionError`, указатель версии 1 от запасного.

9. Приложение. Репозиторий и воспроизведение

```
zk/conf/zoo1.cfg zoo2.cfg zoo3.cfg   конфигурации трёх серверов
zk/apache-zookeeper-3.9.6-bin.tar.gz официальный архив ZooKeeper 3.9.6
zk/env.sh                            JAVA_HOME (Java 17), папка ансамбля, адреса
zk/setup_ensemble.sh                 этап 1: три копии, zoo.cfg и myid
zk/conf/logback.xml                  журнал сервера с дозаписью (история всех перезапусков)
zk/stage2_ensemble.sh                этап 2: роли, потеря одного и двух серверов, перезапуск
zk/ephemeral_demo.py                 этап 2: эфемерный узел в zkCli
coordinator/coordinator.py           координатор: выбор лидера, эпоха, чекпоинт, публикация
coordinator/scenario.py              этапы 3-4: два координатора, SIGSTOP / SIGCONT, итог
coordinator/run_stage3_4.sh          оба сценария + состояние указателя в zkCli
coordinator/hw3/                     код расчёта из ДЗ 3 без изменений
run_all.sh                           полный прогон, все выводы в logs/
logs/stage1_setup.txt                выводы этапа 1
logs/stage2_ensemble.txt             выводы этапа 2
logs/stage2_ephemeral.txt            эфемерный узел
logs/zk-server/zk1..3.log            журналы серверов ZooKeeper
logs/nofence/, logs/fencing/         журналы координаторов этапов 3 и 4, scenario.log, summary.json
logs/failover1..3/                   повторные замеры времени смены лидера
report/                              отчёт (Typst), снимки экрана
```

Листинг 11. Структура репозитория

```
sudo apt install openjdk-17-jdk-headless netcat-openbsd
pip install -r requirements.txt
cp .env.example .env                    # ключи RustFS из ДЗ 2-3, бакет hh-lake с данными ДЗ 3
./run_all.sh                            # этапы 1-4, ~5 минут, все выводы в logs/
```

Листинг 12. Воспроизведение